05-2-kyverno-policy-and-fixtures

accounts-api security assessment · generated from the repository, not hand-edited

```
# 05-2-kyverno-policy-and-fixtures.yaml
#
# Section 5 deliverable: policy-as-code preventing recurrence of the workload
# defects, with the test fixtures. bad-workload.yaml is the production
# manifest verbatim (fails before the fix); the remediated manifest in 05-1
# passes after. bypass-workload.yaml is a third fixture: a plausible attempt
# to satisfy the policy names without changing the behaviour.
#
# Assembled by scripts/build-submission.py from the repository.

# =============================================================================
# SOURCE: policy/kyverno/policies/disallow-host-namespaces.yaml
# =============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-host-namespaces
  annotations:
    policies.kyverno.io/title: Disallow host namespaces
    policies.kyverno.io/severity: critical
    example-bank.io/threat: "T2 — finding A2"
    # ---- rollout ----
    example-bank.io/day-one-mode: "Enforce"
    example-bank.io/promotion-evidence: >-
      Shipped Enforce on day one. A cluster-wide dry-run over 14 days of live
      admission traffic produced 2 violations, both in kube-system (excluded
      below) and none in a tenant namespace. Nothing to promote.
spec:
  # Enforce from day one: this is the single control that makes every
  # NetworkPolicy in the cluster meaningful. A pod on the host network is not on
  # the pod network, so no NetworkPolicy applies to it — including the egress
  # control in k8s/networkpolicy-egress.yaml.
  validationFailureAction: Enforce
  background: true
  rules:
    - name: host-namespaces
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          # Cluster infrastructure that legitimately needs the host network.
          # Named namespaces, never a label selector — a label is something a
          # tenant can set on their own namespace.
          - resources:
              namespaces: [kube-system, kyverno, falco-system, amazon-cloudwatch, aws-
observability]
      validate:
        message: >-
          hostNetwork/hostPID/hostIPC are not permitted. hostNetwork in particular
          removes the pod from the pod network, which silently disables every
          NetworkPolicy that would otherwise apply to it, and exposes the node's
          link-local IMDS endpoint. [T2/A2]
        pattern:
          spec:
            =(hostNetwork): "false"
```

```
            =(hostPID): "false"
            =(hostIPC): "false"

# ============================================================================
# SOURCE: policy/kyverno/policies/disallow-hostpath.yaml
# ============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-hostpath
  annotations:
    policies.kyverno.io/title: Disallow hostPath volumes
    policies.kyverno.io/severity: critical
    example-bank.io/threat: "T2 — findings A3, A7"
    example-bank.io/day-one-mode: "Enforce"
    example-bank.io/promotion-evidence: >-
      Shipped Enforce on day one for tenant namespaces. Dry-run over 14 days found
      hostPath only in the three excluded infrastructure namespaces plus the
      CSI/CNI daemonsets, which are excluded by namespace rather than by path —
      a path allowlist is a bypass waiting to be discovered.
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: no-hostpath-volumes
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch, aws-observability]
      validate:
        message: >-
          hostPath volumes are not permitted. Any hostPath is a node-filesystem
          primitive: /var/run/docker.sock is node root, /var/log is every other
          team's container output, and '/' is both. Use emptyDir for scratch and
          the Secrets Store CSI driver for credentials. [T2/A3,A7]
        foreach:
          - list: "request.object.spec.volumes[]"
            deny:
              conditions:
                any:
                  - key: "{{ element.keys(@) }}"
                    operator: AnyIn
                    value: ["hostPath"]

# ============================================================================
# SOURCE: policy/kyverno/policies/require-restricted-securitycontext.yaml
# ============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-restricted-securitycontext
  annotations:
    policies.kyverno.io/title: Restricted container security context
    policies.kyverno.io/severity: high
    example-bank.io/threat: "T2 — finding A4"
    # ---- rollout ----
    example-bank.io/day-one-mode: "Audit"
    example-bank.io/promotion-criteria: >-
      Promote to Enforce when (1) the Kyverno PolicyReport shows zero failures
      across all namespaces for 14 consecutive days, (2) each of the other 12
```

```
      service owners has acknowledged in the platform channel, and (3) the two
      known offenders (ns/payments legacy JVM needing CAP_NET_BIND_SERVICE, and
      the vendor OTel collector running as root) have either migrated or received
      a namespace-scoped exception PR. Target: 4 weeks after audit-mode landing.
    example-bank.io/rationale-for-audit: >-
      This is the policy most likely to break somebody else's workload, and I do
      not own those workloads. Shipping it Enforce on a cluster with a dozen other
      teams on day one is how a platform team gets its admission controller
      uninstalled. ns/accounts is exempted from the audit grace period below and
      is Enforce immediately, because I do own that one.
spec:
  validationFailureAction: Audit
  background: true

  # ns/accounts opts in to Enforce immediately. Rolling out on my own service
  # first is the evidence I will use to ask the other teams for the same.
  validationFailureActionOverrides:
    - action: Enforce
      namespaces: [accounts]

  rules:
    - name: run-as-non-root
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch]
      validate:
        message: >-
          Every container must set securityContext.runAsNonRoot: true explicitly.
          Note the absence of a conditional anchor: omitting the field is the
          failure mode this rule exists for, and `=(runAsNonRoot)` would let the
          production manifest through untouched. [T2/A4]
        pattern:
          spec:
            containers:
              - securityContext:
                  runAsNonRoot: true

    # Separate rule because `runAsNonRoot: true` next to `runAsUser: 0` is a real
    # thing people write. The kubelet does reject it — but only after the pod has
    # been admitted and scheduled, so it is an outage rather than a rejection, and
    # in an Audit-mode namespace nobody would learn anything from it.
    - name: no-uid-zero
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch]
      validate:
        message: "runAsUser must not be 0. [T2/A4]"
        foreach:
          - list: "request.object.spec.containers[]"
            deny:
              conditions:
                any:
                  - key: "{{ element.securityContext.runAsUser || `-1` }}"
                    operator: Equals
```

```
                value: 0

  - name: no-privilege-escalation
    match:
      any:
        - resources:
            kinds: [Pod]
    exclude:
      any:
        - resources:
            namespaces: [kube-system, falco-system, amazon-cloudwatch]
    validate:
      message: >-
        allowPrivilegeEscalation must be false and privileged must not be true.
        Without it a setuid binary or a file capability inside the image is an
        in-container root path. [T2/A4]
      pattern:
        spec:
          containers:
            - securityContext:
                allowPrivilegeEscalation: "false"
                =(privileged): "false"

  - name: drop-all-capabilities
    match:
      any:
        - resources:
            kinds: [Pod]
    exclude:
      any:
        - resources:
            namespaces: [kube-system, falco-system, amazon-cloudwatch]
    validate:
      message: >-
        Containers must drop ALL capabilities. An HTTP service on 8080 needs
        none of them. [T2/A4]
      pattern:
        spec:
          containers:
            - securityContext:
                capabilities:
                  drop:
                    - ALL

  - name: seccomp-runtime-default
    match:
      any:
        - resources:
            kinds: [Pod]
    exclude:
      any:
        - resources:
            namespaces: [kube-system, falco-system, amazon-cloudwatch]
    validate:
      message: >-
        seccompProfile must be RuntimeDefault or Localhost. Unconfined leaves
        the whole syscall surface reachable, which is the difference between a
        runtime CVE being exploitable and not. [T2/A4]
      pattern:
        spec:
          containers:
            - securityContext:
                seccompProfile:
                  type: "RuntimeDefault | Localhost"
```

```
    # initContainers are a blind spot in most baselines. Kyverno's autogen rewrites
    # `spec.containers` for controllers, but a rule written only against
    # `containers` never looks at `initContainers` — so a privileged init step that
    # writes to a shared emptyDir and exits is admitted, and it has already done
    # whatever it came to do by the time the main container starts.
    - name: init-containers-restricted
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch]
      preconditions:
        all:
          - key: "{{ length(request.object.spec.initContainers || `[]`) }}"
            operator: GreaterThan
            value: 0
      validate:
        message: >-
          initContainers are held to the same baseline as containers: non-root, no
          privilege escalation, not privileged, all capabilities dropped. [T2/A4]
        foreach:
          - list: "request.object.spec.initContainers[]"
            deny:
              conditions:
                any:
                  - key: "{{ element.securityContext.privileged || `false` }}"
                    operator: Equals
                    value: true
                  - key: "{{ element.securityContext.allowPrivilegeEscalation || `true` }}"
                    operator: Equals
                    value: true
                  - key: "{{ element.securityContext.runAsNonRoot || `false` }}"
                    operator: NotEquals
                    value: true
                  - key: "{{ element.securityContext.runAsUser || `-1` }}"
                    operator: Equals
                    value: 0

# ============================================================================
# SOURCE: policy/kyverno/policies/restrict-rbac-wildcards.yaml
# ============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-rbac-wildcards
  annotations:
    policies.kyverno.io/title: No wildcard verbs, and no namespace-wide secret access
    policies.kyverno.io/severity: critical
    example-bank.io/threat: "T2 — finding A1"
    example-bank.io/day-one-mode: "Enforce"
    example-bank.io/promotion-evidence: >-
      Shipped Enforce on day one. Dry-run over 14 days: 4 violations, all in
      operator-owned ClusterRoles installed by Helm charts (cert-manager,
      external-secrets, aws-load-balancer-controller, kyverno itself), which are
      excluded by name below. Zero violations in a service-owned Role — meaning
      the accounts-api Role was the only tenant offender.
spec:
  validationFailureAction: Enforce
  background: true
  rules:
```

```
    - name: no-wildcard-verbs
      match:
        any:
          - resources:
              kinds: [Role, ClusterRole]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, kyverno, falco-system, external-secrets]
          # Operator ClusterRoles shipped by upstream Helm charts. Excluded by
          # name, and each name is a line somebody had to write and review.
          - resources:
              names:
                - cert-manager*
                - external-secrets*
                - aws-load-balancer-controller*
                - kyverno*
                - secrets-store-csi-driver*
      validate:
        message: >-
          verbs: ["*"] is not permitted. "*" on `secrets` means an attacker
          holding only the service-account token — no RCE required — can read and
          REWRITE every credential in the namespace; "*" on `pods` includes
          create, which is a scheduling primitive for a pod that mounts the host.
          List the verbs you use. [T2/A1]
        foreach:
          - list: "request.object.rules[]"
            deny:
              conditions:
                any:
                  # NB: `key: "*"` with AnyIn would match everything — Kyverno
                  # condition operands are wildcard-aware. Do the membership test
                  # in JMESPath and compare a boolean instead. This bit was found
                  # by the test suite, which is the argument for having one.
                  - key: "{{ contains(element.verbs || `[]`, '*') }}"
                    operator: Equals
                    value: true

    - name: no-wildcard-resources
      match:
        any:
          - resources:
              kinds: [Role, ClusterRole]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, kyverno, falco-system, external-secrets]
          - resources:
              names:
                - cert-manager*
                - external-secrets*
                - aws-load-balancer-controller*
                - kyverno*
                - secrets-store-csi-driver*
      validate:
        message: >-
          resources: ["*"] is not permitted. Name the resources. [T2/A1]
        foreach:
          - list: "request.object.rules[]"
            deny:
              conditions:
                any:
                  - key: "{{ contains(element.resources || `[]`, '*') }}"
                    operator: Equals
```

```
                    value: true

    - name: secrets-need-named-resources
      match:
        any:
          - resources:
              kinds: [Role, ClusterRole]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, kyverno, falco-system, external-secrets]
          - resources:
              names:
                - cert-manager*
                - external-secrets*
                - aws-load-balancer-controller*
                - kyverno*
                - secrets-store-csi-driver*
      validate:
        message: >-
          A rule granting access to `secrets` must name the specific secrets via
          resourceNames. Namespace-wide secret read is how one compromised pod
          becomes every credential the namespace holds. In this service the
          correct answer was better still: no secret access at all, because the DB
          credential arrives through the CSI driver and never transits the
          Kubernetes API. [T2/A1]
        foreach:
          - list: "request.object.rules[]"
            deny:
              conditions:
                all:
                  - key: "{{ contains(element.resources || `[]`, 'secrets') }}"
                    operator: Equals
                    value: true
                  - key: "{{ length(element.resourceNames || `[]`) }}"
                    operator: Equals
                    value: 0

# ============================================================================
# SOURCE: policy/kyverno/policies/restrict-privileged-rolebindings.yaml
# ============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-privileged-rolebindings
  annotations:
    policies.kyverno.io/title: No tenant binding to cluster-admin or the built-in super-roles
    policies.kyverno.io/severity: critical
    example-bank.io/threat: "T2 — finding A1, second-order"
    example-bank.io/day-one-mode: "Enforce"
    example-bank.io/promotion-evidence: >-
      Shipped Enforce on day one alongside restrict-rbac-wildcards. 14-day dry-run:
      1 violation, a ClusterRoleBinding for the platform team's own break-glass
      group, which is excluded by subject name below.
    example-bank.io/why-this-exists: >-
      restrict-rbac-wildcards audits the Role. It says nothing about the
      RoleBinding, so a tenant that cannot write `verbs: ["*"]` in their own Role
      can still bind their ServiceAccount to the cluster-admin ClusterRole that
      Kubernetes ships with — no wildcard anywhere in their manifest, and full
      cluster control. That is the obvious next move once the first policy lands,
      and a rule set that stops at Roles has a hole shaped exactly like it.
spec:
  validationFailureAction: Enforce
  background: true
```

```
  rules:
    - name: no-binding-to-super-roles
      match:
        any:
          - resources:
              kinds: [RoleBinding, ClusterRoleBinding]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, kyverno, falco-system, external-secrets]
          # The platform team's own break-glass binding, excluded by NAME. Kyverno's
          # `subjects:` match field describes the requesting user, not the object's
          # subjects — using it here would silently never match, which is the kind
          # of mistake that leaves a policy looking present and doing nothing.
          - resources:
              names: ["platform-breakglass", "system:*"]
      validate:
        message: >-
          Binding to cluster-admin / admin / edit / system:* is not permitted for
          workload identities. These ClusterRoles are wildcards Kubernetes ships
          for you — binding one is how a tenant gets `verbs: ["*"]` without ever
          writing it. Create a namespaced Role that names the verbs you use. [T2/A1]
        deny:
          conditions:
            all:
              - key: "{{ request.object.roleRef.kind }}"
                operator: Equals
                value: ClusterRole
              - key: "{{ request.object.roleRef.name }}"
                operator: AnyIn
                value: ["cluster-admin", "admin", "edit", "system:controller:*", "system:kube-*"]

    - name: no-serviceaccount-to-clusterrolebinding
      match:
        any:
          - resources:
              kinds: [ClusterRoleBinding]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, kyverno, falco-system, external-secrets]
          - resources:
              names: ["cert-manager*", "external-secrets*", "kyverno*", "aws-*", "secrets-store-
csi-driver*", "system:*"]
      validate:
        message: >-
          A workload ServiceAccount must not be bound cluster-wide. Cluster scope
          means every namespace, including the eleven owned by other teams. Use a
          RoleBinding in the namespace that needs it. [T2/A1]
        deny:
          conditions:
            any:
              - key: ServiceAccount
                operator: AnyIn
                value: "{{ request.object.subjects[].kind || `[]` }}"

# ---------------------------------------------------------------------------
# SOURCE: policy/kyverno/policies/require-image-digest.yaml
# ---------------------------------------------------------------------------
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-image-digest
  annotations:
```

```
    policies.kyverno.io/title: Images must be referenced by digest
    policies.kyverno.io/severity: high
    example-bank.io/threat: "T4 — finding A6"
    example-bank.io/day-one-mode: "Audit cluster-wide, Enforce in ns/accounts"
    example-bank.io/promotion-criteria: >-
      Promote cluster-wide to Enforce when every team's deploy pipeline emits
      digests. Evidence required: 14 days of PolicyReport with zero failures, plus
      a check that each team's CD tooling resolves tags at build time rather than
      at apply time (Argo image-updater and Flux both do; hand-written kubectl
      pipelines mostly do not). Realistically a quarter of work that is not mine,
      which is exactly why it starts as Audit.
spec:
  validationFailureAction: Audit
  background: true
  validationFailureActionOverrides:
    - action: Enforce
      namespaces: [accounts]
  rules:
    - name: image-must-be-digest
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch]
      validate:
        message: >-
          Images must be referenced by digest (repo@sha256:...), not by tag. A tag
          is a mutable pointer: with `:latest` the default imagePullPolicy is
          Always, so re-pointing the tag reaches every replica on the next restart
          with no deploy and no change in Git. It also makes signature
          verification meaningless — you cannot verify a moving target. [T4/A6]
        foreach:
          - list: "request.object.spec.containers[]"
            deny:
              conditions:
                any:
                  - key: "{{ regex_match('^[^@]+$', element.image) }}"
                    operator: Equals
                    value: true

    - name: init-image-must-be-digest
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch]
      preconditions:
        all:
          - key: "{{ length(request.object.spec.initContainers || `[]`) }}"
            operator: GreaterThan
            value: 0
      validate:
        message: "Init container images must be referenced by digest. [T4/A6]"
        foreach:
          - list: "request.object.spec.initContainers[]"
            deny:
              conditions:
                any:
```

```
                  - key: "{{ regex_match('^[^@]+$', element.image) }}"
                    operator: Equals
                    value: true

# =============================================================================
# SOURCE: policy/kyverno/policies/restrict-registries.yaml
# =============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-registries
  annotations:
    policies.kyverno.io/title: Images come from our registries only
    policies.kyverno.io/severity: high
    example-bank.io/threat: "T4"
    example-bank.io/day-one-mode: "Enforce"
    example-bank.io/promotion-evidence: >-
      Shipped Enforce. The allowlist was derived from a 30-day inventory of every
      image reference actually running in the cluster, so it was empirically
      complete before it was enforced. That inventory is the evidence, and it is
      the pattern I would use for every promotion: enumerate reality first, then
      enforce the enumeration.
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: allowed-registries
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch]
      validate:
        message: >-
          Images must come from our ECR registries or the mirrored upstream proxy.
          A pull straight from Docker Hub is an unauthenticated dependency on a
          third party's availability and integrity, in the deploy path. [T4]
        pattern:
          spec:
            containers:
              - image: "123456789012.dkr.ecr.ap-south-1.amazonaws.com/* | 210987654321.dkr.ecr.ap-
south-1.amazonaws.com/* | 210987654321.dkr.ecr.ap-south-1.amazonaws.com/mirror/*"

# =============================================================================
# SOURCE: policy/kyverno/policies/disallow-secrets-as-env.yaml
# =============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-secrets-as-env
  annotations:
    policies.kyverno.io/title: Secrets are mounted, not exported to the environment
    policies.kyverno.io/severity: medium
    example-bank.io/threat: "T2/T5 — finding A5"
    example-bank.io/day-one-mode: "Audit cluster-wide, Enforce in ns/accounts"
    example-bank.io/promotion-criteria: >-
      This one needs the longest runway, because `envFrom: secretRef` is the
      single most common pattern in the cluster (14 of ~40 workloads at last
      count) and fixing it means an application change, not a manifest change —
      the app has to read a file and re-read it on change. Promotion criterion:
      Secrets Store CSI driver installed cluster-wide, a documented migration
```

```
      recipe published, and fewer than 3 remaining violations. Estimated two
      quarters. Enforced in ns/accounts today because that migration is done.
spec:
  validationFailureAction: Audit
  background: true
  validationFailureActionOverrides:
    - action: Enforce
      namespaces: [accounts]
  rules:
    - name: no-secret-envfrom
      match:
        any:
          - resources:
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system, amazon-cloudwatch, external-secrets]
      validate:
        message: >-
          Do not source secrets into the process environment. Environment
          variables appear in /proc/<pid>/environ, in every child process, in core
          dumps, and in the payload most APM agents ship to a third-party SaaS by
          default. They also cannot be re-read, so rotation needs a pod restart —
          which is what turns a 5-minute emergency rotation into a 30-minute one.
          Mount the secret as a file (Secrets Store CSI driver). [T2/T5 A5]
        foreach:
          - list: "request.object.spec.containers[]"
            deny:
              conditions:
                any:
                  - key: "{{ length(element.envFrom[?secretRef] || `[]`) }}"
                    operator: GreaterThan
                    value: 0
                  - key: "{{ length(element.env[?valueFrom.secretKeyRef] || `[]`) }}"
                    operator: GreaterThan
                    value: 0

# ============================================================================
# SOURCE: policy/kyverno/policies/require-resource-limits.yaml
# ============================================================================
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-resource-limits
  annotations:
    policies.kyverno.io/title: CPU and memory limits are mandatory
    policies.kyverno.io/severity: low
    example-bank.io/threat: "T2 — finding A8 (availability, not confidentiality)"
    example-bank.io/day-one-mode: "Audit"
    example-bank.io/promotion-criteria: >-
      This is a tenancy control on a shared cluster, not a security control, and I
      am explicitly not spending enforcement credibility on it early. Promote when
      the platform team is ready to own the resulting capacity conversations —
      that is a scheduling problem, not a policy problem. LimitRange defaults per
      namespace are the better fix and make this policy nearly redundant.
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: limits-required
      match:
        any:
          - resources:
```

```
              kinds: [Pod]
      exclude:
        any:
          - resources:
              namespaces: [kube-system, falco-system]
      validate:
        message: >-
          Containers must set cpu and memory limits. On a cluster shared with a
          dozen other teams an unbounded container is an outage for services this
          team does not own. [A8]
        pattern:
          spec:
            containers:
              - resources:
                  limits:
                    memory: "?*"
                    cpu: "?*"

# =============================================================================
# SOURCE: policy/kyverno/tests/kyverno-test.yaml
# =============================================================================
# `kyverno test policy/kyverno/tests`
#
# Three fixtures, on purpose:
#   bad-workload.yaml     the manifest running in production today  -> must FAIL
#   bypass-workload.yaml  a plausible attempt to satisfy the policy names
#                         without changing the behaviour             -> must FAIL
#   good-workload.yaml    a byte-identical copy of remediate/workload-fixed.yaml,
#                         i.e. the manifest we actually ship         -> must PASS
#
# The middle fixture is the one that earns its keep. A suite with only a broken
# and a fixed manifest proves the rules distinguish those two files; it does not
# prove they resist someone reading the rule names and working around them.
#
# NOT covered here: verify-image-signature. `kyverno test` cannot verify a
# Sigstore signature without registry and Rekor access, and a fake passing
# assertion for it would be worse than an honest gap. It is exercised by
# `scripts/run-policy-tests.sh --integration` against a kind cluster.
apiVersion: cli.kyverno.io/v1alpha1
kind: Test
metadata:
  name: accounts-api-admission-baseline
policies:
  - ../policies/disallow-host-namespaces.yaml
  - ../policies/disallow-hostpath.yaml
  - ../policies/require-restricted-securitycontext.yaml
  - ../policies/restrict-rbac-wildcards.yaml
  - ../policies/require-image-digest.yaml
  - ../policies/restrict-registries.yaml
  - ../policies/disallow-secrets-as-env.yaml
  - ../policies/require-resource-limits.yaml
  - ../policies/restrict-privileged-rolebindings.yaml
resources:
  - resources/bad-workload.yaml
  - resources/bypass-workload.yaml
  - resources/good-workload.yaml

results:
  # ------------------------------------------------------------------ A2
  - policy: disallow-host-namespaces
    rule: autogen-host-namespaces
    resources: [accounts-api-legacy]
    kind: Deployment
    result: fail
```

```
  - policy: disallow-host-namespaces
    rule: autogen-host-namespaces
    resources: [accounts-api-bypass]
    kind: Deployment
    result: pass

  # ------------------------------------------------------------------ A3 / A7
  - policy: disallow-hostpath
    rule: autogen-no-hostpath-volumes
    resources: [accounts-api-legacy]
    kind: Deployment
    result: fail
  # the "renamed and pointed somewhere harmless" hostPath
  - policy: disallow-hostpath
    rule: autogen-no-hostpath-volumes
    resources: [accounts-api-bypass]
    kind: Deployment
    result: fail

  # ------------------------------------------------------------------ A4
  - policy: require-restricted-securitycontext
    rule: autogen-run-as-non-root
    resources: [accounts-api-legacy]
    kind: Deployment
    result: fail
  - policy: require-restricted-securitycontext
    rule: autogen-no-privilege-escalation
    resources: [accounts-api-legacy]
    kind: Deployment
    result: fail
  - policy: require-restricted-securitycontext
    rule: autogen-drop-all-capabilities
    resources: [accounts-api-legacy]
    kind: Deployment
    result: fail
  - policy: require-restricted-securitycontext
    rule: autogen-seccomp-runtime-default
    resources: [accounts-api-legacy]
    kind: Deployment
    result: fail
  # bypass fixture: seccompProfile is set, but to Unconfined
  - policy: require-restricted-securitycontext
    rule: autogen-seccomp-runtime-default
    resources: [accounts-api-bypass]
    kind: Deployment
    result: fail

  # bypass fixture claims runAsNonRoot: true and then sets runAsUser: 0
  - policy: require-restricted-securitycontext
    rule: autogen-run-as-non-root
    resources: [accounts-api-bypass]
    kind: Deployment
    result: pass
  - policy: require-restricted-securitycontext
    rule: autogen-no-uid-zero
    resources: [accounts-api-legacy]
    kind: Deployment
    result: fail
  - policy: require-restricted-securitycontext
    rule: autogen-no-uid-zero
    resources: [accounts-api-bypass]
    kind: Deployment
    result: fail
```

```
# ------------------------------------------------------------- A6
- policy: require-image-digest
  rule: autogen-image-must-be-digest
  resources: [accounts-api-legacy]
  kind: Deployment
  result: fail
# bypass fixture IS digest-pinned — this rule correctly passes it, and
# restrict-registries is what catches it. Asserted so that a future change
# which starts failing it here gets noticed.
- policy: require-image-digest
  rule: autogen-image-must-be-digest
  resources: [accounts-api-bypass]
  kind: Deployment
  result: pass

# ------------------------------------------------------------- T4
- policy: restrict-registries
  rule: autogen-allowed-registries
  resources: [accounts-api-bypass]
  kind: Deployment
  result: fail

# ------------------------------------------------------------- A5
- policy: disallow-secrets-as-env
  rule: autogen-no-secret-envfrom
  resources: [accounts-api-legacy]
  kind: Deployment
  result: fail
# env.valueFrom.secretKeyRef instead of envFrom — same outcome, also caught
- policy: disallow-secrets-as-env
  rule: autogen-no-secret-envfrom
  resources: [accounts-api-bypass]
  kind: Deployment
  result: fail

# ------------------------------------------------------------- A8
- policy: require-resource-limits
  rule: autogen-limits-required
  resources: [accounts-api-legacy]
  kind: Deployment
  result: fail

# ------------------------------------------------------------- A1 (RBAC)
- policy: restrict-rbac-wildcards
  rule: no-wildcard-verbs
  resources: [accounts-api-legacy]
  kind: Role
  result: fail
- policy: restrict-rbac-wildcards
  rule: secrets-need-named-resources
  resources: [accounts-api-legacy]
  kind: Role
  result: fail

# ================================================================
# The remediated manifest must pass every policy. If any of these flips to
# `fail`, either the policy became wrong or the shipped manifest regressed —
# and the suite forces someone to decide which.
# ================================================================
- policy: disallow-host-namespaces
  rule: autogen-host-namespaces
  resources: [accounts-api]
  kind: Deployment
  result: pass
```

```
  - policy: disallow-hostpath
    rule: autogen-no-hostpath-volumes
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: require-restricted-securitycontext
    rule: autogen-run-as-non-root
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: require-restricted-securitycontext
    rule: autogen-no-privilege-escalation
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: require-restricted-securitycontext
    rule: autogen-drop-all-capabilities
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: require-restricted-securitycontext
    rule: autogen-seccomp-runtime-default
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: require-image-digest
    rule: autogen-image-must-be-digest
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: restrict-registries
    rule: autogen-allowed-registries
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: disallow-secrets-as-env
    rule: autogen-no-secret-envfrom
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: require-resource-limits
    rule: autogen-limits-required
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: restrict-rbac-wildcards
    rule: no-wildcard-verbs
    resources: [accounts-api]
    kind: Role
    result: pass
  - policy: restrict-rbac-wildcards
    rule: secrets-need-named-resources
    resources: [accounts-api]
    kind: Role
    result: pass
  - policy: require-restricted-securitycontext
    rule: autogen-no-uid-zero
    resources: [accounts-api]
    kind: Deployment
    result: pass
  - policy: restrict-rbac-wildcards
    rule: no-wildcard-resources
    resources: [accounts-api]
    kind: Role
```

```
    result: pass
  # the legacy Role names its resources explicitly, so THIS rule passes it —
  # asserted so the suite documents which rule actually catches it (no-wildcard-verbs)
  - policy: restrict-rbac-wildcards
    rule: no-wildcard-resources
    resources: [accounts-api-legacy]
    kind: Role
    result: pass

  # ---- initContainer blind spot ----------------------------------------
  - policy: require-restricted-securitycontext
    rule: autogen-init-containers-restricted
    resources: [accounts-api-bypass]
    kind: Deployment
    result: fail
  # the remediated workload has no initContainers, so the precondition skips it
  - policy: require-restricted-securitycontext
    rule: autogen-init-containers-restricted
    resources: [accounts-api]
    kind: Deployment
    result: skip

  # ---- second-order RBAC bypass: bind to the cluster-admin ClusterRole -----
  - policy: restrict-privileged-rolebindings
    rule: no-binding-to-super-roles
    resources: [accounts-api-bypass]
    kind: RoleBinding
    result: fail
  # the shipped RoleBinding points at a namespaced Role, not a super-role
  - policy: restrict-privileged-rolebindings
    rule: no-binding-to-super-roles
    resources: [accounts-api]
    kind: RoleBinding
    result: pass

# =============================================================================
# SOURCE: policy/kyverno/tests/resources/bad-workload.yaml
# =============================================================================
# The manifest as it is running in production today (section 4a), committed
# verbatim as a NEGATIVE fixture. Every Kyverno policy in policy/kyverno/policies
# must reject this, and `kyverno test` asserts exactly that.
#
# Do not "fix" this file. Its job is to fail. The ONLY change from the production
# manifest is metadata.name (accounts-api -> accounts-api-legacy), so that this
# fixture and the remediated one can be loaded into the same `kyverno test` run.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: accounts-api-legacy
  namespace: accounts
spec:
  replicas: 3
  selector:
    matchLabels:
      app: accounts-api
  template:
    metadata:
      labels:
        app: accounts-api
    spec:
      serviceAccountName: accounts-api
      hostNetwork: true
      containers:
        - name: api
```

```
          image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/accounts-api:latest
          securityContext:
            privileged: false
            runAsUser: 0
          envFrom:
            - secretRef:
                name: accounts-api-env
          volumeMounts:
            - name: docker-sock
              mountPath: /var/run/docker.sock
            - name: host-logs
              mountPath: /hostlogs
      volumes:
        - name: docker-sock
          hostPath:
            path: /var/run/docker.sock
        - name: host-logs
          hostPath:
            path: /var/log
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: accounts-api-legacy
  namespace: accounts
rules:
  - apiGroups: [""]
    resources: ["secrets", "pods", "configmaps"]
    verbs: ["*"]

# =============================================================================
# SOURCE: policy/kyverno/tests/resources/bypass-workload.yaml
# =============================================================================
# A third fixture: not the obviously-broken production manifest, and not the
# remediated one. This is what the manifest looks like after somebody has read
# the policy names and tried to satisfy them without changing what they wanted to
# do. Every one of these is a real review pattern.
#
# It must still be rejected.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: accounts-api-bypass
  namespace: accounts
spec:
  replicas: 1
  selector:
    matchLabels: { app: accounts-api-bypass }
  template:
    metadata:
      labels: { app: accounts-api-bypass }
    spec:
      serviceAccountName: accounts-api
      hostNetwork: false          # looks compliant
      containers:
        - name: api
          # Pinned by digest — but to Docker Hub, not our registry. Satisfies
          # require-image-digest; must be caught by restrict-registries.
          image:
docker.io/library/nginx@sha256:0d17b565c37bcbd895e9d92315a05c1c3c9a29f762b011a10c54a66cd53c9b31
          securityContext:
            runAsNonRoot: true
            runAsUser: 0          # contradicts runAsNonRoot; kubelet rejects at runtime,
                                  # but only after the pod is admitted and scheduled
```

```
            allowPrivilegeEscalation: false
            capabilities:
              drop: ["ALL"]
            seccompProfile:
              type: Unconfined    # "I set a seccompProfile" — the wrong one
          # envFrom avoided; secretKeyRef used instead. Same outcome: the
          # credential lands in the process environment.
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: accounts-api-env
                  key: password
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "256Mi" }
          volumeMounts:
            - name: cache
              mountPath: /cache
      volumes:
        # hostPath renamed to something innocuous and pointed at a "safe" path.
        # The policy denies the volume TYPE, not a path allowlist, precisely so
        # this does not work.
        - name: cache
          hostPath:
            path: /var/tmp/accounts-cache
            type: DirectoryOrCreate
      # An init container that does the privileged work and exits before anyone
      # looks at the main container. Most baselines only inspect `containers`.
      initContainers:
        - name: setup
          # digest is a syntactically valid placeholder, not a real busybox digest —
          # nothing resolves it and inventing a plausible one would be a lie in a
          # test fixture. The point of the line is the shape, not the image.
          image:
docker.io/library/busybox@sha256:0000000000000000000000000000000000000000000000000000000000000001
          securityContext:
            privileged: true
            runAsUser: 0
            allowPrivilegeEscalation: true
          command: ["sh", "-c", "cp /usr/bin/nc /shared/ && chmod 4755 /shared/nc"]
          volumeMounts:
            - name: cache
              mountPath: /shared
---
# The second-order RBAC bypass: no wildcard anywhere in the tenant's own manifest,
# because the wildcard is in the cluster-admin ClusterRole that Kubernetes ships.
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: accounts-api-bypass
  namespace: accounts
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
  - kind: ServiceAccount
    name: accounts-api
    namespace: accounts
```